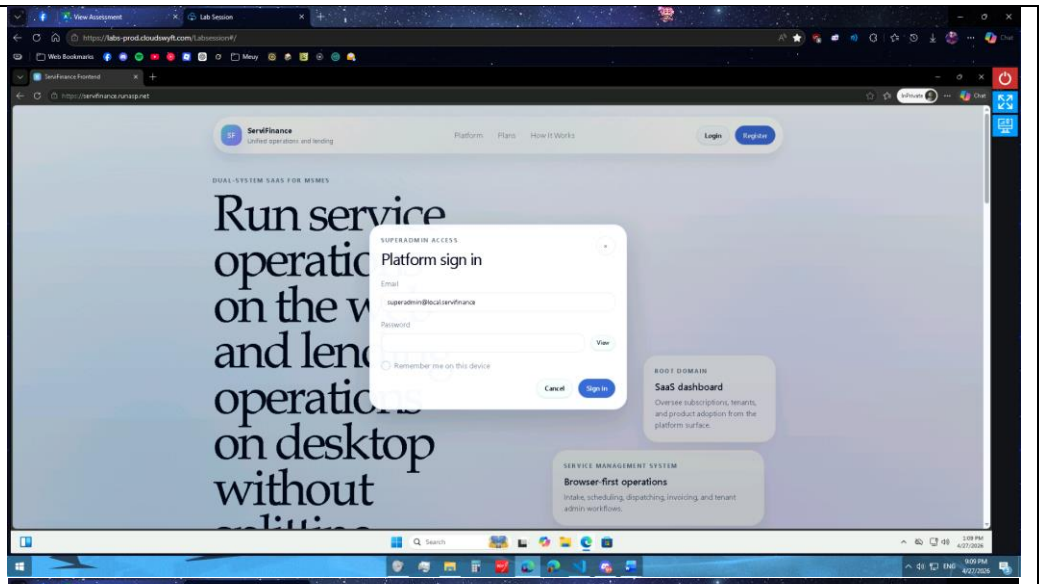
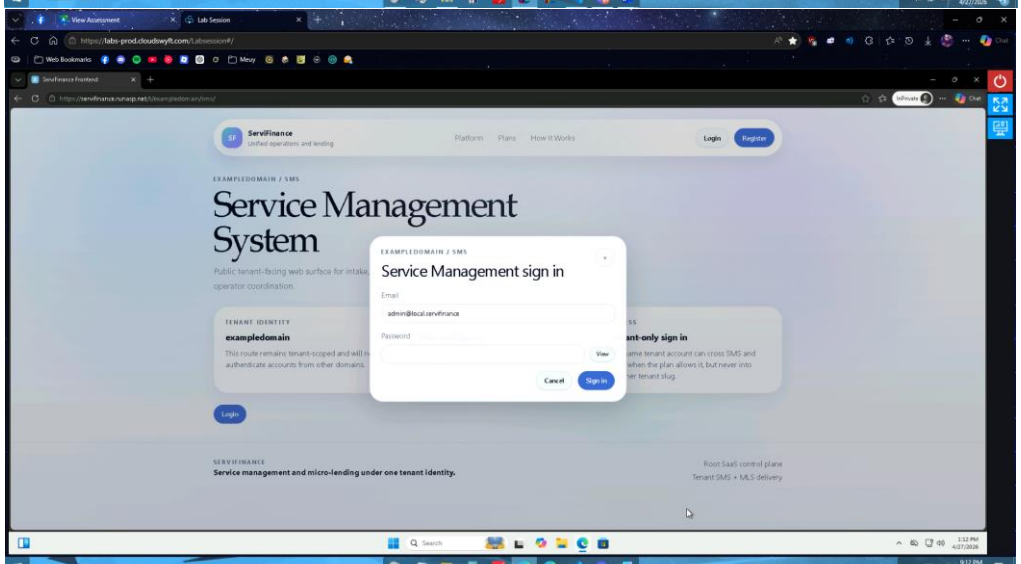
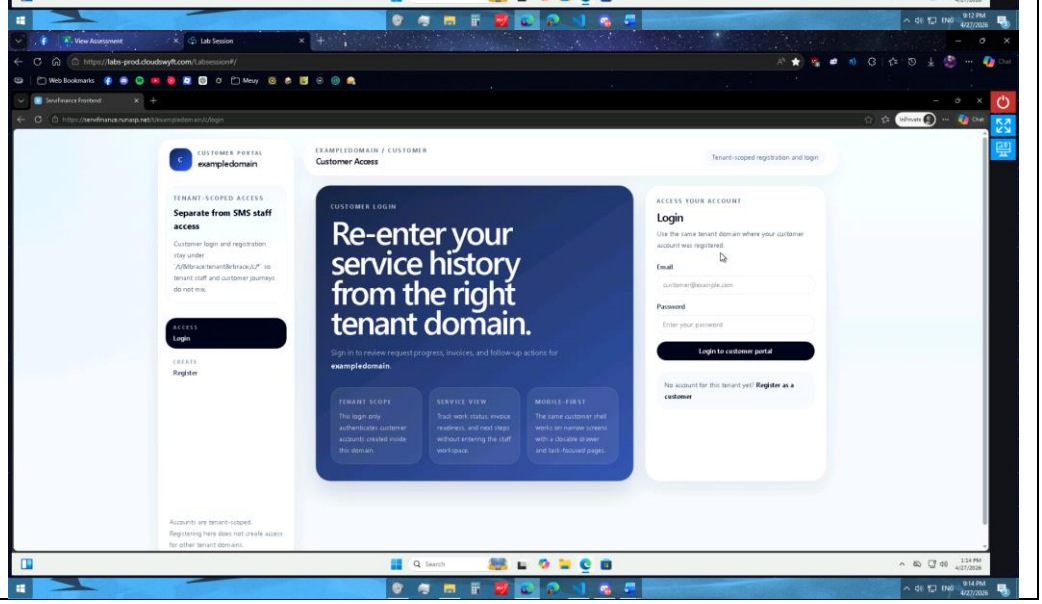
NAME	**CHRISTOPHER LLANDER L. VILLACINO**
PROJECT TITLE	**ServiFinance: A Multi-Tenant SaaS Service Management System with Embedded Micro-Lending**
SUBJECT: **CODE:** **TIME:**	**IT15/L Integrative Programming and Technologies** **8466** **1:30 – 3:30 PM**
TOPIC (Type of Business Process)	**#33 Service Management System**
Products/Services	**SaaS platform for Service-Oriented MSMEs**
Deployed Link / Website link (it should be HTTPS)	**https://servifinance.runasp.net**
Role Based-Access	**Multi-Tenancy** **Super Admin Acct.** **Username: superadmin@servifinance.com** **Password: SuperAdmin123!** **Admin Acct.** **Username: admin@exampledomain.com** **Password: ExampleAdmin123!** **Client Acct.** **Username: n/a** **Password: n/a**
3rd Deliverables-1st Hands-on Exam **Deployed Transactions (CRUD)**	

Log in-Page (Screenshot)	  

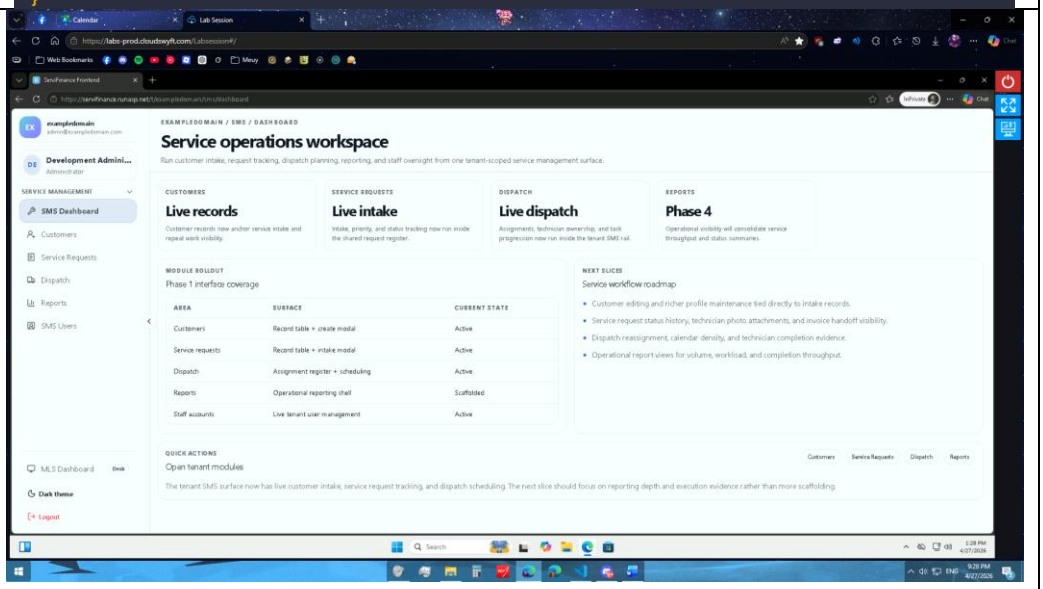
	Login pages from SuperAdmin root, Tenant domain & Customer portal.
Log in-Page - Source Code (Screenshot)	

```
namespace ServiFinance.Api.Endpoints;

using System.Security.Claims;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using ServiFinance.Api.Infrastructure;
using ServiFinance.Application.Auth;
using ServiFinance.Infrastructure.Configuration;
using static ServiFinance.Api.Infrastructure.ProgramEndpointSupport;

0 references
internal static class AuthApiEndpointMappings {
  1 reference
  public static RouteGroupBuilder MapAuthApiEndpoints(this RouteGroupBuilder api, SessionTokenOptions sessionTokenOptions) {
    var authApi = api.MapGroup("/auth");
    authApi.MapPost("/root/login", [AllowAnonymous] async Task<IResult> (
        HttpContext httpContext,
        [FromBody] RootApiLoginRequest request,
        IUserAuthenticationService authenticationService,
        ISessionTokenService sessionTokenService) => {
          var user = await authenticationService.AuthenticateAsync(
          new AuthenticationRequest(request.Email, request.Password, AuthenticationSurface.Root),
          httpContext.RequestAborted);
          if (user is null) {
            return Results.Unauthorized();
          }

          var tokens = await sessionTokenService.CreateSessionAsync(user, AuthenticationSurface.Root, request.RememberMe,
          httpContext.RequestAborted);
          if (request.UseCookieSession) {
            await SignInUserAsync(httpContext, user, request.RememberMe);
            WriteRefreshTokenCookie(httpContext, tokens.RefreshToken, request.RememberMe ? TimeSpan.FromDays
            (sessionTokenOptions.PersistentRefreshTokenDays) : null);
          }

          return Results.Ok(new AuthSessionResponse(tokens, ToCurrentSessionUser(user, AuthenticationSurface.Root)));
        });
    authApi.MapPost("/tenant/login", [AllowAnonymous] async Task<IResult> (
        HttpContext httpContext,
        [FromBody] TenantApiLoginRequest request,
        IUserAuthenticationService authenticationService,
        ISessionTokenService sessionTokenService) => {
          var isMls = string.Equals(request.TargetSystem, "mls", StringComparison.OrdinalIgnoreCase);
          var surface = isMls ? AuthenticationSurface.TenantDesktop : AuthenticationSurface.TenantWeb;
          var tenantSlug = NormalizeTenantSlug(request.TenantDomainSlug);
          var user = await authenticationService.AuthenticateAsync(
```

```
            new AuthenticationRequest(request.Email, request.Password, surface, tenantSlug),
            httpContext.RequestAborted);
            if (user is null) {
              return Results.Unauthorized();
            }

            var tokens = await sessionTokenService.CreateSessionAsync(user, surface, cancellationToken: httpContext.
            RequestAborted);
            if (request.UseCookieSession) {
              await SignInUserAsync(httpContext, user);
              WriteRefreshTokenCookie(httpContext, tokens.RefreshToken);
            }

            return Results.Ok(new AuthSessionResponse(tokens, ToCurrentSessionUser(user, surface)));
          });

        authApi.MapPost("/customer/login", [AllowAnonymous] async Task<IResult> (
            HttpContext httpContext,
            [FromBody] CustomerApiLoginRequest request,
            ICustomerAuthenticationService authenticationService,
            ISessionTokenService sessionTokenService) => {
              var tenantSlug = NormalizeTenantSlug(request.TenantDomainSlug);
              if (string.IsNullOrWhiteSpace(tenantSlug)) {
                return Results.BadRequest(new { error = "Invalid tenant domain slug" });
              }
              var user = await authenticationService.AuthenticateAsync(
                  request.Email, request.Password, tenantSlug, httpContext.RequestAborted);
              if (user is null) {
                return Results.Unauthorized();
              }

              var surface = AuthenticationSurface.CustomerWeb;
              var tokens = await sessionTokenService.CreateSessionAsync(user, surface, cancellationToken: httpContext.
              RequestAborted);
              if (request.UseCookieSession) {
                await SignInUserAsync(httpContext, user);
                WriteRefreshTokenCookie(httpContext, tokens.RefreshToken);
              }

              return Results.Ok(new AuthSessionResponse(tokens, ToCurrentSessionUser(user, surface)));
            });

        authApi.MapPost("/customer/register", [AllowAnonymous] async Task<IResult> (
            HttpContext httpContext,
            [FromBody] CustomerRegisterRequest request,
            ICustomerAuthenticationService authenticationService,
            ISessionTokenService sessionTokenService) => {
              try {
                var user = await authenticationService.RegisterAsync(request, httpContext.RequestAborted);

                var surface = AuthenticationSurface.CustomerWeb;
                var tokens = await sessionTokenService.CreateSessionAsync(user, surface, cancellationToken: httpContext.
                RequestAborted);
                if (request.UseCookieSession) {
                  await SignInUserAsync(httpContext, user);
                  WriteRefreshTokenCookie(httpContext, tokens.RefreshToken);
                }

                return Results.Ok(new AuthSessionResponse(tokens, ToCurrentSessionUser(user, surface)));
              }
              catch (InvalidOperationException ex) {
                return Results.BadRequest(new { error = ex.Message });
              }
            });

        authApi.MapGet("/refresh", [AllowAnonymous] async Task<IResult> (
            HttpContext httpContext,
            ISessionTokenService sessionTokenService) =>
            await RefreshSessionAsync(httpContext, null, sessionTokenService));
        authApi.MapPost("/refresh", [AllowAnonymous] async Task<IResult> (
            HttpContext httpContext,
            [FromBody] RefreshSessionRequest? request,
            ISessionTokenService sessionTokenService) =>
            await RefreshSessionAsync(httpContext, request, sessionTokenService));
        authApi.MapPost("/logout", [AllowAnonymous] async Task<IResult> (
            HttpContext httpContext,
            [FromBody] RefreshSessionRequest? request,
            ISessionTokenService sessionTokenService) => {
              var refreshToken = request?.RefreshToken ?? ReadRefreshTokenCookie(httpContext);
              if (!string.IsNullOrWhiteSpace(refreshToken)) {
                await sessionTokenService.RevokeSessionAsync(refreshToken, httpContext.RequestAborted);
              }

              DeleteRefreshTokenCookie(httpContext);
              await httpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
              return Results.NoContent();
            });
        authApi.MapGet("/me", [Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)] (ClaimsPrincipal
        principal) => {
          var userId = Guid.Parse(principal.FindFirstValue(ClaimTypes.NameIdentifier)!);
          var tenantId = Guid.Parse(principal.FindFirstValue("tenant_id")!);
          var tenantDomainSlug = principal.FindFirstValue("tenant_domain_slug") ?? string.Empty;
```

```
        var fullName = principal.FindFirstValue(ClaimTypes.Name) ?? string.Empty;
        var email = principal.FindFirstValue(ClaimTypes.Email) ?? string.Empty;
        _ = Enum.TryParse<AuthenticationSurface>(principal.FindFirstValue("surface"), true, out var surface);
        var roles = principal.FindAll(ClaimTypes.Role).Select(claim => claim.Value).Distinct(StringComparer.OrdinalIgnoreCase).
        ToArray();

        return Results.Ok(new CurrentSessionUser(userId, tenantId, tenantDomainSlug, email, fullName, roles, surface));
      });

      return authApi;
    }
  }
```

Dashboard (Client/End-User) (Screenshot)



Tenant-specific Dashboard. Shows dashboard necessary for the tenant owner of this service. Currently unfinished.

Dashboard - Source Code (Client/End-User) (Screenshot)

```
namespace ServiFinance.Api.Endpoints.TenantSms;

using Microsoft.EntityFrameworkCore;
using static ServiFinance.Api.Infrastructure.ProgramEndpointSupport;

internal static class TenantSmsReportsEndpointMappings {
  public static RouteGroupBuilder MapTenantSmsReportsEndpoints(this RouteGroupBuilder tenantApi) {
    tenantApi.MapGet("/sms/reports/overview", async Task<IResult> (
        HttpContext httpContext,
        string tenantDomainSlug,
        DateTime? dateFrom,
        DateTime? dateTo,
        ServiFinance.Infrastructure.Data.ServiFinanceDbContext dbContext,
        CancellationToken cancellationToken) => {
          if (!IsTenantRouteAllowed(httpContext.User, tenantDomainSlug)) {
            return Results.Forbid();
          }

          var utcToday = DateTime.UtcNow.Date;
          var utcTomorrow = utcToday.AddDays(1);
          var reportEndExclusive = (dateTo?.Date ?? utcToday).AddDays(1);
          var reportStartInclusive = dateFrom?.Date ?? reportEndExclusive.AddDays(-7);
          if (reportEndExclusive <= reportStartInclusive) {
            return Results.BadRequest(new { error = "Report end date must be on or after the start date." });
          }

          var reportDayCount = (int)(reportEndExclusive - reportStartInclusive).TotalDays;
          var previousReportEndExclusive = reportStartInclusive;
          var previousReportStartInclusive = previousReportEndExclusive.AddDays(-reportDayCount);

          var customerCount = await dbContext.Customers.CountAsync(cancellationToken);
          var serviceRequestCount = await dbContext.ServiceRequests.CountAsync(cancellationToken);
          var activeAssignmentCount = await dbContext.Assignments.CountAsync(
              entity => entity.AssignmentStatus == "Scheduled" || entity.AssignmentStatus == "In Progress" ||
entity.AssignmentStatus == "On Hold",
              cancellationToken);
          var completedAssignmentCount = await dbContext.Assignments.CountAsync(
              entity => entity.AssignmentStatus == "Completed",
              cancellationToken);

          var dailyActivity = new {
            NewCustomersToday = await dbContext.Customers.CountAsync(
                entity => entity.CreatedAtUtc >= utcToday && entity.CreatedAtUtc < utcTomorrow,
                cancellationToken),
            NewRequestsToday = await dbContext.ServiceRequests.CountAsync(
                entity => entity.CreatedAtUtc >= utcToday && entity.CreatedAtUtc < utcTomorrow,
                cancellationToken),
            AssignmentsScheduledToday = await dbContext.Assignments.CountAsync(
                entity => entity.CreatedAtUtc >= utcToday && entity.CreatedAtUtc < utcTomorrow,
                cancellationToken),
            AssignmentsCompletedToday = await dbContext.AssignmentEvents.CountAsync(
                entity => entity.AssignmentStatus == "Completed" &&
                    entity.CreatedAtUtc >= utcToday &&
```

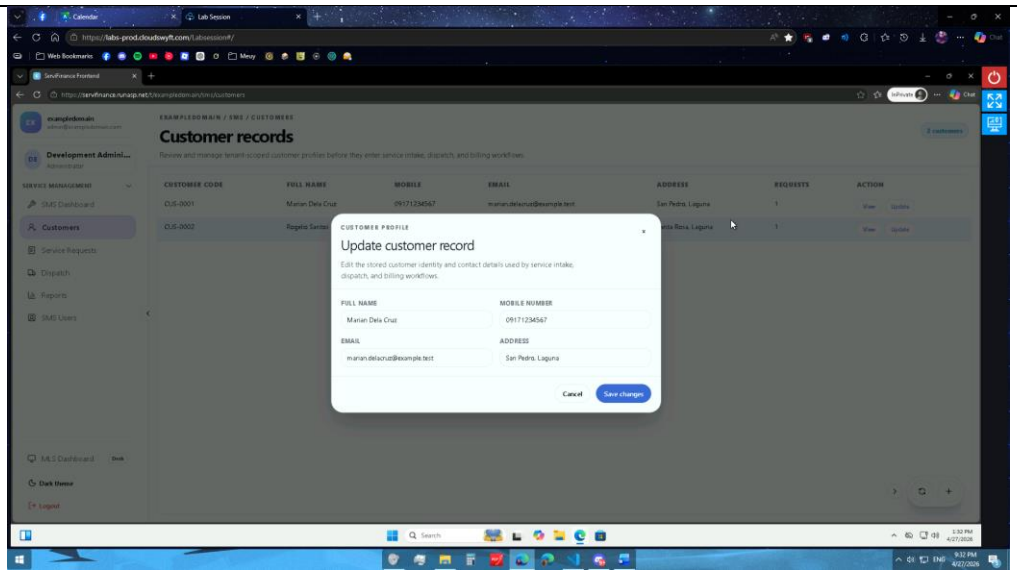
```
                    entity.CreatedAtUtc < utcTomorrow,
                cancellationToken)
        };

        var serviceStatusDistribution = await dbContext.ServiceRequests
            .AsNoTracking()
            .GroupBy(entity => entity.CurrentStatus)
            .Select(group => new {
              Status = group.Key,
              Count = group.Count()
            })
            .OrderByDescending(item => item.Count)
            .ThenBy(item => item.Status)
            .ToListAsync(cancellationToken);

        var technicianWorkload = await dbContext.Assignments
            .AsNoTracking()
            .GroupBy(entity => new {
              entity.AssignedUserId,
              FullName = entity.AssignedUser!.FullName
            })
            .Select(group => new {
              UserId = group.Key.AssignedUserId,
              group.Key.FullName,
              ActiveAssignments = group.Count(entity => entity.AssignmentStatus == "In Progress"),
              ScheduledAssignments = group.Count(entity => entity.AssignmentStatus == "Scheduled"),
              CompletedAssignments = group.Count(entity => entity.AssignmentStatus == "Completed")
            })
            .OrderByDescending(item => item.ActiveAssignments)
            .ThenByDescending(item => item.ScheduledAssignments)
            .ThenBy(item => item.FullName)
            .ToListAsync(cancellationToken);

        var customers = await dbContext.Customers
            .AsNoTracking()
            .Select(entity => new CustomerCreatedAt(entity.CreatedAtUtc))
            .ToListAsync(cancellationToken);

        var serviceRequests = await dbContext.ServiceRequests
            .AsNoTracking()
            .Select(entity => new ReportServiceRequest(
                entity.Id,
                entity.CreatedAtUtc,
                entity.RequestedServiceDate,
                entity.CurrentStatus))
            .ToListAsync(cancellationToken);

        var assignments = await dbContext.Assignments
            .AsNoTracking()
            .Select(entity => new ReportAssignment(
                entity.ServiceRequestId,
                entity.CreatedAtUtc,
                entity.ScheduledStartUtc,
                entity.ScheduledEndUtc,
                entity.AssignmentStatus))
            .ToListAsync(cancellationToken);

        var completedRequestLogs = await dbContext.StatusLogs
            .AsNoTracking()
            .Where(entity => entity.Status == "Completed")
            .GroupBy(entity => entity.ServiceRequestId)
            .Select(group => new CompletedRequestLog(
                group.Key,
                group.Min(entity => entity.ChangedAtUtc)))
            .ToListAsync(cancellationToken);

        var invoices = await dbContext.Invoices
            .AsNoTracking()
            .Where(entity => entity.ServiceRequestId != null)
            .Select(entity => new ServiceInvoice(entity.InvoiceDateUtc))
            .ToListAsync(cancellationToken);

        var firstScheduledLookup = assignments
            .Where(entity => entity.ScheduledStartUtc.HasValue)
            .GroupBy(entity => entity.ServiceRequestId)
            .ToDictionary(
                group => group.Key,
                group => group.Min(entity => entity.ScheduledStartUtc!.Value));
        var completionLookup = completedRequestLogs.ToDictionary(entity => entity.ServiceRequestId, entity =>
entity.CompletedAtUtc);

        var currentWindowActivity = CreateWindowActivity(
            reportStartInclusive,
            reportEndExclusive,
            customers,
            serviceRequests,
            assignments,
            completedRequestLogs,
            invoices);
        var previousWindowActivity = CreateWindowActivity(
            previousReportStartInclusive,
            previousReportEndExclusive,
            customers,
            serviceRequests,
            assignments,
            completedRequestLogs,
            invoices);

        var comparison = new[] {
          CreateComparisonMetric("new-customers", "New customers", currentWindowActivity.NewCustomers,
previousWindowActivity.NewCustomers),
          CreateComparisonMetric("new-requests", "New service requests", currentWindowActivity.NewRequests,
previousWindowActivity.NewRequests),
          CreateComparisonMetric("assignments-scheduled", "Assignments scheduled",
currentWindowActivity.AssignmentsScheduled, previousWindowActivity.AssignmentsScheduled),
          CreateComparisonMetric("assignments-completed", "Assignments completed",
currentWindowActivity.AssignmentsCompleted, previousWindowActivity.AssignmentsCompleted),
```

```
            CreateComparisonMetric("completed-requests", "Completed requests",
currentWindowActivity.CompletedRequests, previousWindowActivity.CompletedRequests),
            CreateComparisonMetric("invoices-finalized", "Invoices finalized",
currentWindowActivity.InvoicesFinalized, previousWindowActivity.InvoicesFinalized)
          };

          var completedRequestsInWindow = serviceRequests
              .Where(entity => completionLookup.TryGetValue(entity.Id, out var completedAtUtc) &&
                  completedAtUtc >= reportStartInclusive &&
                  completedAtUtc < reportEndExclusive)
              .Select(entity => new CompletedRequestTurnaround(
                  entity.CreatedAtUtc,
                  completionLookup[entity.Id],
                  firstScheduledLookup.TryGetValue(entity.Id, out var scheduledStartUtc)
                    ? scheduledStartUtc
                    : (DateTime?)null))
              .ToList();

          var turnaround = new {
            CompletedRequests = completedRequestsInWindow.Count,
            AverageIntakeToCompletionHours = CalculateAverageHours(
                completedRequestsInWindow,
                entity => entity.CreatedAtUtc,
                entity => entity.CompletedAtUtc),
            AverageRequestToScheduleHours = CalculateAverageHours(
                completedRequestsInWindow.Where(entity => entity.FirstScheduledStartUtc.HasValue),
                entity => entity.CreatedAtUtc,
                entity => entity.FirstScheduledStartUtc),
            AverageScheduledWorkHours = CalculateAverageHours(
                assignments.Where(entity =>
                    entity.AssignmentStatus == "Completed" &&
                    entity.ScheduledStartUtc.HasValue &&
                    entity.ScheduledEndUtc.HasValue &&
                    entity.ScheduledEndUtc.Value >= reportStartInclusive &&
                    entity.ScheduledEndUtc.Value < reportEndExclusive),
                entity => entity.ScheduledStartUtc,
                entity => entity.ScheduledEndUtc),
            OverdueOpenRequests = serviceRequests.Count(entity =>
                entity.CurrentStatus != "Completed" &&
                entity.RequestedServiceDate.HasValue &&
                entity.RequestedServiceDate.Value.Date < utcToday)
          };

          var catalog = new[] {
            new {
              Key = "daily-activity",
              Title = "Daily Activity Summary",
              Scope = "Customers, intake, and dispatch",
              Freshness = "Live",
              Owner = "Tenant operations",
              Description = "Tracks what entered the tenant workflow today across customer intake, service
requests, and dispatch."
            },
            new {
              Key = "service-status-distribution",
              Title = "Service Status Distribution",
              Scope = "Service request register",
              Freshness = "Live",
              Owner = "Service management",
              Description = "Shows how work is distributed across queued, scheduled, in-service, and completed
states."
            },
            new {
              Key = "technician-workload",
              Title = "Technician Workload",
              Scope = "Assignment register",
              Freshness = "Live",
              Owner = "Dispatch",
              Description = "Summarizes active, scheduled, and completed assignments per assigned staff member."
            },
            new {
              Key = "window-comparison",
              Title = "Window Comparison",
              Scope = "Selected period versus previous period",
              Freshness = "Live",
              Owner = "Business owner",
              Description = "Compares the selected operating window against the immediately preceding period of
equal length."
            },
            new {
              Key = "turnaround-metrics",
              Title = "Turnaround Metrics",
              Scope = "Intake-to-completion and scheduling efficiency",
              Freshness = "Live",
              Owner = "Service operations",
              Description = "Measures completion pace, scheduling lead time, work duration, and overdue open
requests."
            }
          };

          return Results.Ok(new {
            Catalog = catalog,
            DailyActivity = dailyActivity,
            ServiceStatusDistribution = serviceStatusDistribution,
            TechnicianWorkload = technicianWorkload,
            ReportingWindow = new {
              DateFromUtc = reportStartInclusive,
              DateToUtc = reportEndExclusive.AddDays(-1),
              PreviousDateFromUtc = previousReportStartInclusive,
              PreviousDateToUtc = previousReportEndExclusive.AddDays(-1),
              DayCount = reportDayCount
            },
            WindowedActivity = currentWindowActivity,
            Comparison = comparison,
            Turnaround = turnaround,
            Totals = new {
              Customers = customerCount,
```

```
                ServiceRequests = serviceRequestCount,
                ActiveAssignments = activeAssignmentCount,
                CompletedAssignments = completedAssignmentCount
            }
        });
    });

    return tenantApi;
  }

  private static WindowActivity CreateWindowActivity(
      DateTime startInclusive,
      DateTime endExclusive,
      IReadOnlyCollection<CustomerCreatedAt> customers,
      IReadOnlyCollection<ReportServiceRequest> serviceRequests,
      IReadOnlyCollection<ReportAssignment> assignments,
      IReadOnlyCollection<CompletedRequestLog> completedRequestLogs,
      IReadOnlyCollection<ServiceInvoice> invoices) {
    return new WindowActivity(
        customers.Count(entity => entity.CreatedAtUtc >= startInclusive && entity.CreatedAtUtc < endExclusive),
        serviceRequests.Count(entity => entity.CreatedAtUtc >= startInclusive && entity.CreatedAtUtc <
endExclusive),
        assignments.Count(entity => entity.CreatedAtUtc >= startInclusive && entity.CreatedAtUtc < endExclusive),
        assignments.Count(entity =>
            entity.AssignmentStatus == "Completed" &&
            entity.ScheduledEndUtc.HasValue &&
            entity.ScheduledEndUtc.Value >= startInclusive &&
            entity.ScheduledEndUtc.Value < endExclusive),
        completedRequestLogs.Count(entity => entity.CompletedAtUtc >= startInclusive && entity.CompletedAtUtc <
endExclusive),
        invoices.Count(entity => entity.InvoiceDateUtc >= startInclusive && entity.InvoiceDateUtc <
endExclusive));
  }

  private static object CreateComparisonMetric(string key, string label, int currentValue, int previousValue) {
    var deltaValue = currentValue - previousValue;
    double? deltaPercentage = previousValue == 0
        ? currentValue == 0 ? 0d : null
        : Math.Round((double)deltaValue / previousValue * 100d, 1);

    return new {
      Key = key,
      Label = label,
      CurrentValue = currentValue,
      PreviousValue = previousValue,
      DeltaValue = deltaValue,
      DeltaPercentage = deltaPercentage
    };
  }

  private static double? CalculateAverageHours<T>(
      IEnumerable<T> source,
      Func<T, DateTime?> startSelector,
      Func<T, DateTime?> endSelector) {
    var durations = source
        .Select(entity => new {
          Start = startSelector(entity),
          End = endSelector(entity)
        })
        .Where(entity => entity.Start.HasValue && entity.End.HasValue && entity.End.Value >= entity.Start.Value)
        .Select(entity => (entity.End!.Value - entity.Start!.Value).TotalHours)
        .ToList();

    return durations.Count == 0 ? null : Math.Round(durations.Average(), 1);
  }

  private sealed record CustomerCreatedAt(DateTime CreatedAtUtc);

  private sealed record ReportServiceRequest(
      Guid Id,
      DateTime CreatedAtUtc,
      DateTime? RequestedServiceDate,
      string CurrentStatus);

  private sealed record ReportAssignment(
      Guid ServiceRequestId,
      DateTime CreatedAtUtc,
      DateTime? ScheduledStartUtc,
      DateTime? ScheduledEndUtc,
      string AssignmentStatus);

  private sealed record CompletedRequestLog(Guid ServiceRequestId, DateTime CompletedAtUtc);

  private sealed record ServiceInvoice(DateTime InvoiceDateUtc);

  private sealed record CompletedRequestTurnaround(
      DateTime CreatedAtUtc,
      DateTime CompletedAtUtc,
      DateTime? FirstScheduledStartUtc);

  private sealed record WindowActivity(
      int NewCustomers,
      int NewRequests,
      int AssignmentsScheduled,
      int AssignmentsCompleted,
      int CompletedRequests,
      int InvoicesFinalized);
}
```

Dashboard reuses some of functions from Reports endpoint.

Update/Edit Page (Screenshot)



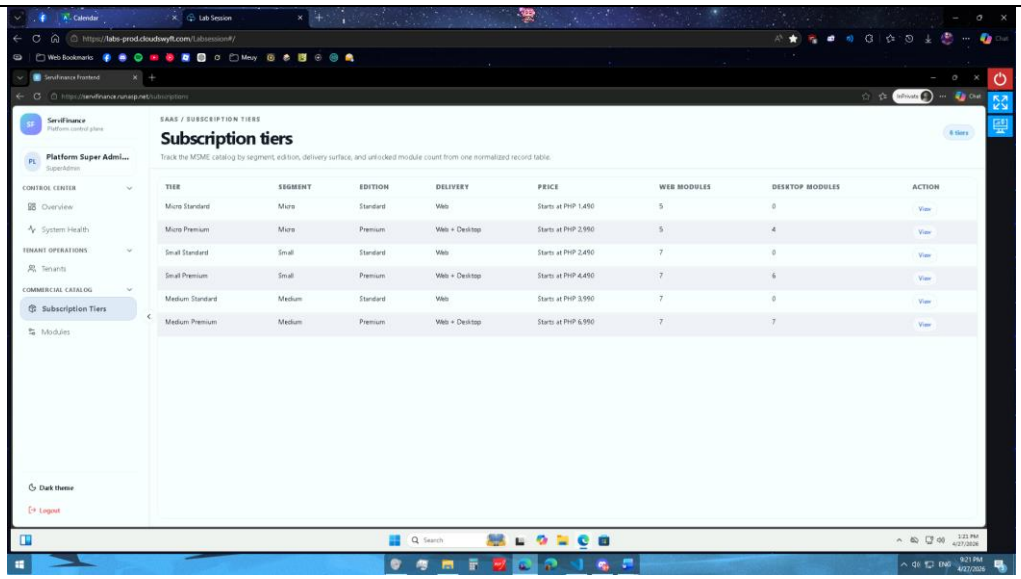
Update Existing Customer Record modal. Update existing customer identity.

Update/Edit Page - Source Code (Screenshot)

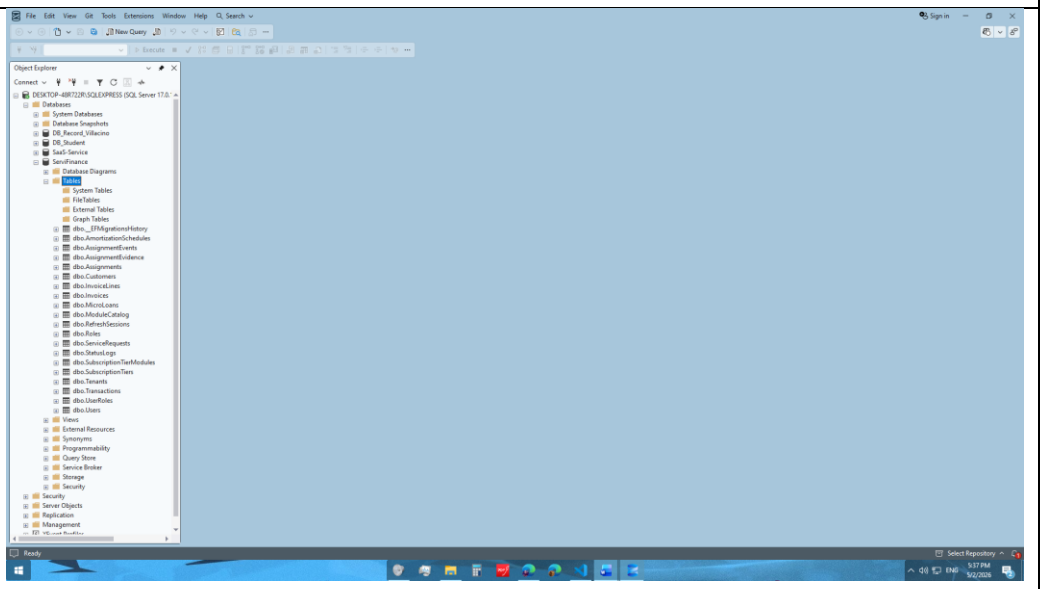
```
namespace ServiFinance.Api.Endpoints.TenantSms;

using System.Security.Claims;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ServiFinance.Api.Contracts;
using ServiFinance.Application.Auth;
using static ServiFinance.Api.Infrastructure.ProgramEndpointSupport;

internal static class TenantSmsCustomersEndpointMappings {
  private const int EmailMaxLength = 50;

  public static RouteGroupBuilder MapTenantSmsCustomersEndpoints(this RouteGroupBuilder tenantApi) {
    tenantApi.MapGet("/sms/customers", async Task<IResult> (…);
    tenantApi.MapPost("/sms/customers", async Task<IResult> (…);
    tenantApi.MapPut("/sms/customers/{customerId:guid}", async Task<IResult> (
        HttpContext httpContext,
        string tenantDomainSlug,
        Guid customerId,
        [FromBody] CreateCustomerRecordRequest request,
        ServiFinance.Infrastructure.Data.ServiFinanceDbContext dbContext,
        CancellationToken cancellationToken) => {
          if (!IsTenantRouteAllowed(httpContext.User, tenantDomainSlug)) {
            return Results.Forbid();
          }

          if (string.IsNullOrWhiteSpace(request.FullName)) {
            return Results.BadRequest(new { error = "Customer full name is required." });
          }
          if (request.Email.Trim().Length > EmailMaxLength) {
            return Results.BadRequest(new { error = $"Customer email must be {EmailMaxLength} characters or
fewer." });
          }

          var customer = await dbContext.Customers
          .Include(entity => entity.ServiceRequests)
          .SingleOrDefaultAsync(entity => entity.Id == customerId, cancellationToken);
          if (customer is null) {
            return Results.NotFound();
          }

          customer.FullName = request.FullName.Trim();
          customer.MobileNumber = request.MobileNumber.Trim();
          customer.Email = request.Email.Trim();
          customer.Address = request.Address.Trim();

          await dbContext.SaveChangesAsync(cancellationToken);

          return Results.Ok(new {
            customer.Id,
            customer.CustomerCode,
            customer.FullName,
            customer.MobileNumber,
            customer.Email,
            customer.Address,
            customer.CreatedAtUtc,
            ServiceRequestCount = customer.ServiceRequests.Count
          });
        });

    return tenantApi;
  }
}
```

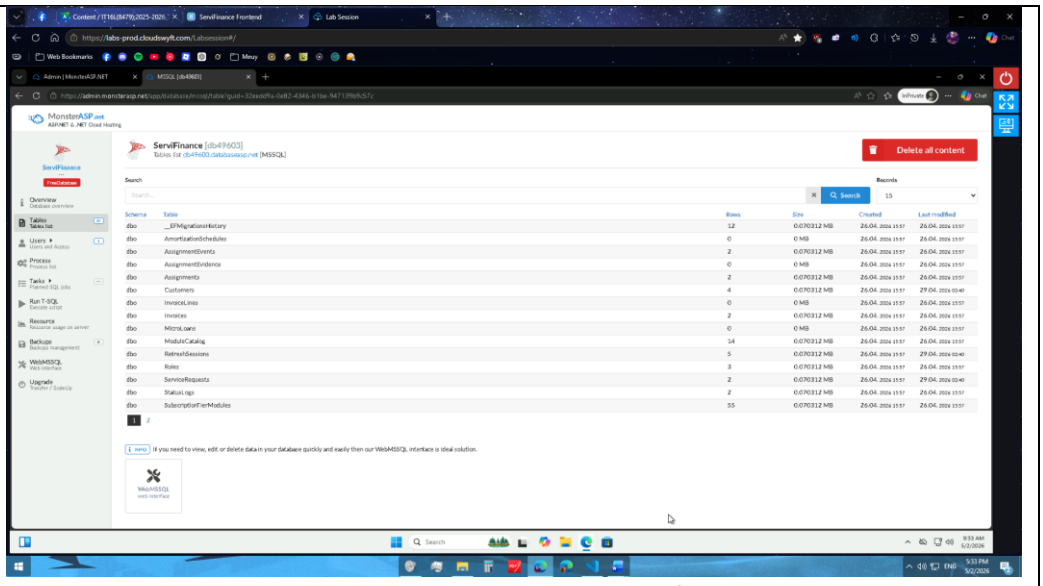
Archive (Screenshot)

Not yet implemented.

Archive – Source Code (Screenshot)	***Not yet implemented.***
Display Records (Screenshot)	 SuperAdmin's Subscription Tiers module, shows the list of tiers for the end-users to choose. Currently cannot add/modify.

Display Records - Source Code (Screenshot)	`api.MapGet("/catalog/subscription-tiers", [AllowAnonymous] async Task<IResult> (`
	`ISubscriptionTierCatalogService subscriptionTierCatalogService,`
	`CancellationToken cancellationToken) =>`
	`Results.Ok(await subscriptionTierCatalogService.GetActiveTiersAsync(cancellationToken)));`

Database – SQL Server (Local) Screenshot	 MSSQL on local machine. **Note: I'm not going to haphazardly setup local SQL database on virtual lab.*

Database – Deployed/Online Screenshot	 MSSQL Database deployed on MonsterASP.net's database hosting.
Date Submitted:	
Teacher's Feedback	_____ _____ _____ _____ _____ _____ _____ _____ Student's Signature (after feedbacking) Teacher's Signature